

```

1  (*
2                                     CS51 Lab 8
3                                     Functors
4                                     Part 1
5  *)
6
7  (*=====
8  Readings:
9
10     This lab builds on material from Chapter 12.5 to the end of Chapter
11     12 of the textbook <http://book.cs51.io>, which should be read
12     before the lab session.
13
14  Objective:
15
16     This lab provides practice in the use of functors.
17
18     This part of the lab has been adapted from the functors chapter of
19     Real World OCaml <http://dev.realworldocaml.org/functors.html>.
20  =====*)
21
22  (*=====
23  Functors - Part 1.
24
25  For the first part of this lab, you will explore a realistic and
26  useful application of functors -- a library to support interval
27  computation.
28
29  Intervals come up in many different contexts. As a concrete example,
30  calendars need to associate events with time intervals (3-4pm or
31  11:30am-3:30pm). Intervals can be empty (like the interval starting at
32  4pm and ending at the previous 3pm; it contains no time at all). We
33  might want to know if a value is contained within an interval. (For
34  example, 4pm is not contained within 11:30am-3:30pm, but is contained
35  within 2-5pm.) We may want to know the intersection of two
36  intervals. (The intersection of 3-4pm and 11:30am-3:30pm is 3-3:30pm.)
37
38  Importantly, intervals can be defined over different data types. For
39  instance you may want to use intervals over floating point values or,
40  as in the example above, intervals of times. But regardless of the
41  type the intervals are over, you'll want some common basic operations:
42  constructing an interval, checking if an interval is empty, checking
43  if a point is contained within an interval, and intersecting two
44  intervals. (Of course, in an actual interval library, you'd want
45  several other operations as well, but this will be enough to get us
46  started.)
47
48  Let's examine how we can build a *generic* interval library using
49  functors.
50
51  An interval is defined by its endpoints. In order to operate on
52  intervals, we need to be able to manipulate the endpoints as well, in

```

```

53 particular by comparing two endpoints for how they are ordered. Thus,
54 to define intervals over a certain kind of endpoints, we'll need the
55 type of the endpoints and a way to compare two endpoints.
56
57 The `ORDERED_TYPE` signature from the textbook (Section 12.3) will
58 serve us nicely here. It uses the OCaml convention for comparison
59 functions used by the `Stdlib.compare` function:
60
61     compare x y < 0      (* x < y *)
62     compare x y = 0      (* x = y *)
63     compare x y > 0      (* x > y *)
64 *)
65
66 module type ORDERED_TYPE =
67   sig
68     type t
69     val compare : t -> t -> int
70   end ;;
71
72   (*****
73   A digression: It can be quite confusing which `compare` function
74   we are referring to in the code in this lab, between the `compare`
75   functions in implementations of the `ORDERED_TYPE` signature and
76   the `compare` function that is always available by virtue of the
77   opening of the `Stdlib` module. To regain some clarity, we'll
78   "turn off" the unqualified `Stdlib` function. *)
79
80   let compare _ = failwith "Use `Stdlib.compare` instead" ;;
81
82   (* You can still use that function, but you'll need to qualify it
83   with its module name as `Stdlib.compare`.
84   *****)
85
86   (*.....
87 Exercise 1A: Complete the following functor for making interval modules.
88
89 We represent an interval with a variant type, which is either `Empty`
90 or `Interval (x, y)`, where `x` and `y` are the bounds of the
91 interval, and are themselves contained within the interval. The
92 representation has an invariant that `x` is always less than or equal
93 to `y` (where the notion of "less than or equal" is given by the
94 `compare` function provided by the `ORDERED_TYPE` that the intervals
95 are built from).
96
97 The functor definition starts out as
98
99     module MakeInterval (Endpoint : ORDERED_TYPE) =
100       struct
101         ...
102       end
103
104 Here, the argument to the `MakeInterval` functor is a module, given
105 the name `Endpoint` and constrained to satisfy the `ORDERED_TYPE`
106 module signature. Thus, we know that `Endpoint` will provide both a

```

```

107 type (`Endpoint.t`) and a comparison function over that type
108 (`Endpoint.compare`). We can and will use those in defining the module
109 being defined by the functor.
110
111 Now, complete the functor definition below.
112 .....*)
113
114 module MakeInterval (Endpoint : ORDERED_TYPE) =
115   struct
116     type interval =
117       | Interval of Endpoint.t * Endpoint.t
118       | Empty
119
120     (* create low high -- Returns a new interval covering `low` to
121      * `high` inclusive. If `low` is greater than `high`, then the
122      * interval is empty. *)
123     let create (low : Endpoint.t) (high : Endpoint.t) : interval =
124       failwith "create not implemented"
125
126     (* is_empty intvl -- Returns true if and only if `intvl` is
127      * empty *)
128     let is_empty (intvl : interval) : bool =
129       failwith "is_empty not implemented"
130
131     (* contains intvl x -- Returns true if and only if the value `x`
132      * is contained within `intvl` *)
133     let contains (intvl : interval) (x : Endpoint.t) : bool =
134       failwith "contains not implemented"
135
136     (* intersect intvl1 intvl2 -- Returns the intersection of `intvl1`
137      * and `intvl2` *)
138     let intersect (intvl1 : interval) (intvl2 : interval) : interval =
139       failwith "intersect not implemented"
140     end ;;
141
142     (*.....
143     Exercise 1B: Using the completed functor above, instantiate an
144     *integer* interval module.
145     .....*)
146
147     module IntInterval =
148       struct end ;; (* <-- replace this line with an appropriate module
149                      definition using the `MakeInterval` functor *)
150
151     (*.....
152     Exercise 1C: Using your newly created integer interval module, create
153     two non-empty intervals named `intvl1` and `intvl2` that have some
154     overlap, and calculate their intersection as `intvl1_intersect_intvl2`.
155     .....*)
156
157     let intvl1 = failwith "not implemented" ;;
158     let intvl2 = failwith "not implemented" ;;
159     let intvl1_intersect_intvl2 = failwith "not implemented" ;;
160

```

```

161 (* There's currently a problem with the `MakeInterval` functor. It's
162 not abstract enough. Notably we are working with an invariant that a
163 valid non-empty interval has an upper bound that is greater than or
164 equal to its lower bound. However, this is only enforced by the
165 `create` function, and as it turns out, we can actually bypass the
166 `create` function due to our lack of an abstraction barrier.
167
168 This expression returns `true`, as expected.
169
170     IntInterval.is_empty (IntInterval.create 4 3) ;;
171
172 This, however, returns `false`. Yikes.
173
174     IntInterval.is_empty (IntInterval.Interval (4, 3)) ;;
175
176 **Make sure you understand why this is a problem. If you don't see the
177 issue, call over a staff member to discuss.**
178
179 To make our functor more abstract, we need to restrict the output of
180 `MakeInterval` to an interface that prevents users from directly
181 creating interval implementations themselves without using the
182 `create` function.
183
184 .....
185 Exercise 2A: Complete the following interface for an interval
186 module. Note in particular that we should add a new type `endpoint` to
187 give us a way of abstractly referring to the type for the endpoints of
188 an interval.
189 .....*)
190
191 module type INTERVAL =
192   sig
193     type interval
194     type endpoint
195     (* ... complete the interface here ... *)
196   end ;;
197
198 (* .....
199 Exercise 2B: Augment the `MakeSafeInterval` functor using the code you
200 wrote for `MakeInterval` as a starting point, such that it returns a
201 module restricted to the `INTERVAL` signature. (Much of the
202 implementation can be copied from `MakeInterval` above.) **Don't
203 forget to specify the module type.**
204 .....*)
205
206 module MakeSafeInterval (Endpoint : ORDERED_TYPE) =
207   struct
208     (* ... complete the module implementation here ... *)
209   end ;;
210
211 (* We have successfully made our returned module abstract, but believe
212 it or not, it is now too abstract. In fact, we have not exposed the
213 type of endpoints to the user, meaning we cannot even create intervals
214 now. The abstraction barrier is too strong. To demonstrate the problem

```

```

215 ...
216
217 .....
218 Exercise 2C: Create an `IntSafeInterval` module using the new
219 `MakeSafeInterval` functor.
220 .....*)
221
222 module IntSafeInterval =
223     struct end ;; (* <-- replace this line with an appropriate module
224                    definition using the `MakeSafeInterval` functor *)
225
226 (* Now, try evaluating the following expression in the REPL:
227
228     IntSafeInterval.create 2 3 ;;
229
230 A type error will appear:
231
232     Error: This expression has type int but an expression was expected of type
233           IntInterval.endpoint
234
235 To make the interface slightly less abstract, we can make use of a
236 sharing constraint, which informs the compiler that a given type
237 *within* the implementation is equal to some other type from *outside*
238 the implementation. In this case, we want to inform the compiler the
239 type of our endpoint is an `int`, and more generally that the type of
240 our endpoint is `Endpoint.t`, where `Endpoint` was the `ORDERED_TYPE`
241 module that is the argument of the functor. We can do so with the
242 following syntax:
243
244     <Module_type> with type <type> = <type'>
245
246 For instance, we can create int interval and float interval interfaces
247 that reveal the type of endpoints as follows: *)
248
249 module type INT_INTERVAL =
250     INTERVAL with type endpoint = int ;;
251
252 module type FLOAT_INTERVAL =
253     INTERVAL with type endpoint = float ;;
254
255 (* Modules that satisfy these interfaces will allow users to actually
256 construct intervals of the desired types.
257
258 While sharing constraints solve the abstraction issue discussed
259 earlier, they now present a new problem. They will result in code
260 duplication in implementation. The solution to this is to use sharing
261 constraints in the MakeInterval functor, exposing that the type of an
262 endpoint is equal to Endpoint.t. The functor can then be used to
263 create interval modules of various types without duplicating
264 code. That's what we'll do next. *)
265
266 (*.....
267 Exercise 3A: Define a new functor `MakeBestInterval`. It should take an
268 ORDERED_TYPE module for the endpoints of the intervals, and return a

```

```

269 module satisfying INTERVAL *with appropriate sharing constraints
270 to allow the creation of generic interval modules*.
271
272 If you do this correctly, your code for `MakeBestInterval` should be
273 almost identical to your code for `MakeSafeInterval` with the
274 exception of any needed sharing constraints.
275 .....*)
276
277 (* ... place your implementation of the MakeBestInterval functor here ... *)
278
279 (* We now have a fully functioning functor that can create interval
280 modules of whatever type we want, with the appropriate abstraction
281 level.
282
283 .....
284 Exercise 3B: Use the `MakeBestInterval` functor to create a new int
285 interval module called `IntBestInterval`, and test that it works as
286 expected.
287
288 You may for instance want to try the problematic lines from Exercise 1C.
289
290 The following expression should still return `true`, as expected:
291
292     IntBestInterval.is_empty (IntBestInterval.create 4 3) ;;
293
294 and this expression should no longer return `false`. What does it
295 return instead?
296
297     IntBestInterval.is_empty (IntBestInterval.Interval (4, 3)) ;;
298 .....*)
299
300 module IntBestInterval = struct end ;;

```